



Povrayst

Declarative raytracing in Typst.

version 0.1.0 · April 2026

github.com/bernsteining/povrayst · typst.app/universe/package/povrayst

Contents

1. Introduction	2
2. Caching and iteration	2
3. Scene Description Language	2
4. Getting started	3
5. Config reference	4
6. Resolution & quality	5
6.1. width / height	5
6.2. quality	5
7. Antialiasing	5
8. Recipes	6
9. Transparent background	6
10. Multi-file scenes (includes)	6
11. Typst-driven scene parameterisation	7
12. Tracing depth & advanced options	7
12.1. max-trace-level	7
12.2. extra (escape hatch)	8
13. Examples	8
13.1. Constructive solid geometry	8
13.2. Quaternion Julia fractal	10
13.3. Clebsch diagonal cubic	11
13.4. Schoen gyroid	12
13.5. Lissajous knot 8_{21}	13
13.6. Hopf fibration	14
13.7. Advanced lighting	15
14. Debugging	16
15. Further reading and other examples	17
16. Render times	17
17. Conclusion	17

1. Introduction

POV-Ray (*Persistence of Vision Raytracer*) is a free, declarative ray tracer first released in 1991. There is no interactive editor: a scene is a plain text file describing camera, lights, geometry, and surfaces, and the renderer turns it into a pixel-perfect image. The same `.pov` source produces the same output on any platform and any version — scenes dif cleanly, parameterise easily, and stay reproducible.

What makes POV-Ray useful for mathematical and scientific figures is the breadth of primitives: implicit algebraic surfaces (cubic, quartic, poly up to degree 15), quaternion fractals (`julia_fractal`), swept tubes (`sphere_sweep`), photon caustics, and function `{ ... }` expressions for arbitrary procedural patterns and isosurfaces. The gallery at the end of this document — Clebsch cubic, Hopf fbration, Lissajous knot, gyroid, caustics — is a few dozen lines of SDL each, no external assets. Povrayst wraps POV-Ray 3.8 as a Typst WebAssembly plugin so all of that renders inline when you compile.

2. Caching and iteration

Typst memoises `render()` by its arguments. Identical calls across compiles return the cached PNG without re-running the plugin — change the scene string or any kwarg and it re-renders; otherwise it's free. This makes incremental documentation editing very fast.

Run `typst watch doc.typ` to pick up changes on save. A cold first compile runs every scene once (typically a few seconds to a couple minutes each, depending on complexity); subsequent edits to prose, layout, or unrelated sections complete in milliseconds.

For expensive renders you iterate on rarely (a high-resolution cover, say), compile once to a separate `.typ` that writes a PNG to disk, then reference that PNG via `image()` in your main document — the logo at the top of this PDF is produced that way, since I needed the PNG for the `README.md`.

3. Scene Description Language

A minimal POV-Ray scene has a camera, at least one `light_source`, and one or more objects:

povray

```
camera { location <0, 2, -5> look_at 0 angle 40 }
light_source { <4, 6, -4> color rgb 1.2 }
sphere { 0, 1 pigment { rgb <1, 0.4, 0.15> } }
```

Objects carry a pigment (colour) and optionally a finish (specular/diffuse/reflection), an interior (index of refraction, dispersion), and a normal (surface bump).

Pigments and normals can be procedural patterns — checker, gradient, spherical, marble, wood, granite, crackle, or arbitrary function `{ x*x + y*y + sin(z) }` expressions — so most textures need no image files at all.

`#declare`, `#while`, `#macro`, and arithmetic on every numeric parameter let you parameterise scenes programmatically; the gallery's Hopf and gyroid examples use this to generate fibres and isosurface functions.

Have a look at the [Scene Description Language's documentation](#) for more information.

4. Getting started

The package exports two functions, `pov` and `render`. Both accept a POV-Ray scene plus the same keyword arguments (full reference in Section 5). They differ only in how the scene is supplied:

- `pov(scene, ..kwargs)` accepts the scene as a **string** or as a fenced povray **raw block**. Use it for inline scenes.
- `render(scene, ..kwargs)` accepts the scene as a **string only**. Use it for scenes loaded from `.pov` files via `read()`.

Internally `pov` extracts the text from a raw block (or passes a string straight through) and forwards to `render`.

```
#import "../pkg/povray.typ": pov, render

// Inline scene with `pov`:
#pov("camera { location <0, 2, -6> look_at 0 angle 35 }
      light_source { <4, 6, -4> rgb 1.2 }
      sphere { 0, 1 pigment { rgb <0.90, 0.40, 0.15> } }",
      width: 480, height: 360)

// Scene from a file with `render`:
#render(read("scene.pov"), width: 480, height: 360)
```

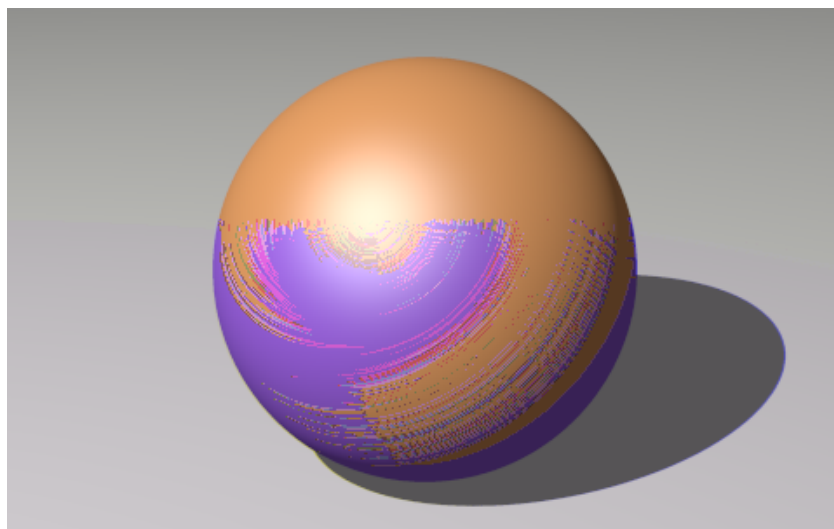


Figure 1: The scene above rendered.

For really terse one-shots, install a `show` rule so that every fenced povray block becomes a render in place:

```
#show raw.where(lang: "povray"): pov
```

After this, any fenced povray block in your document renders directly, with no `#pov(...)` wrapper. Useful to keep syntax highlighting and avoid writing code in a raw string.

```
```povray
camera { location <0, 2, -6> look_at 0 angle 35 }
light_source { <4, 6, -4> rgb 1.2 }
sphere { 0, 1 pigment { rgb <0.90, 0.40, 0.15> } }
```

...

## 5. Config reference

Every keyword argument to `render()`, its default, and the POV-Ray option it maps to. The option names in the right column come verbatim from POV-Ray's `.ini` / command-line reference — see the [official command-line options](#) for the full semantics of each flag. Also check [POV-Ray's documentation](#).

kwarg	default	POV-Ray option	effect
<b>Includes</b>			
<code>includes</code>	<code>(:)</code>	Typst-side <code>#include</code> expansion	dictionary {name: content} spliced before parsing
<b>Output resolution</b>			
<code>width</code>	800	<code>Width</code>	image width in pixels
<code>height</code>	600	<code>Height</code>	image height in pixels
<b>Quality</b>			
<code>quality</code>	9	<code>Quality (0-11)</code>	0 ambient only, 2 +shadows, 5 +refection, 9 full radiosity
<b>Antialiasing</b>			
<code>antialias</code>	true	<code>Antialias=on/off</code>	master switch for adaptive supersampling
<code>aa-threshold</code>	0.3	<code>Antialias_Threshold</code>	colour delta that triggers extra samples; lower = smoother
<code>aa-method</code>	2	<code>Sampling_Method (1/2/3)</code>	1 fixed grid, 2 adaptive recursive, 3 generic oversampling (3.8+)
<code>aa-depth</code>	3	<code>Antialias_Depth (1-9)</code>	recursion depth; up to depth <sup>2</sup> samples/pixel
<code>aa-jitter</code>	true	<code>Jitter=on/off</code>	randomise sub-pixel positions to break up aliasing
<code>aa-jitter-amount</code>	none	<code>Jitter_Amount</code>	jitter magnitude 0.0–1.0
<code>aa-gamma</code>	none	<code>Antialias_Gamma</code>	gamma used when comparing sub-sample colours
<b>Gamma</b>			
<code>display-gamma</code>	none	<code>Display_Gamma</code>	gamma the final image is rendered <i>for</i>
<code>file-gamma</code>	none	<code>File_Gamma</code>	gamma assumed for <code>rgb &lt;...&gt;</code> colour literals in the scene
<b>Tracing</b>			
<code>max-trace-level</code>	none	<code>Max_Trace_Level</code>	cap on refraction / refraction ray depth
<code>bounding</code>	none	<code>Bounding=on/off</code>	toggle automatic bounding-slab acceleration
<code>bounding-threshold</code>	none	<code>Bounding_Threshold</code>	minimum children before an auto-bound is built
<code>remove-bounds</code>	none	<code>Remove_Bounds=on/off</code>	discard user bounded_by when POV-Ray's own bound is tighter
<code>split-unions</code>	none	<code>Split_Unions=on/off</code>	split non-overlapping union children into separate bounds
<b>Partial render</b>			
<code>start-row</code>	none	<code>Start_Row</code>	render only this pixel-row range
<code>end-row</code>	none	<code>End_Row</code>	(integers ≥ 1 or fractions 0..1)
<code>start-column</code>	none	<code>Start_Column</code>	pixel-column range
<code>end-column</code>	none	<code>End_Column</code>	
<b>Output encoding</b>			
<code>output-alpha</code>	false	<code>Output_Alpha=on/off</code>	emit RGBA PNG; use with background { color rgbt <...>,1> }
<code>compression</code>	none	<code>Compression (0-9)</code>	PNG deflate level, 0 none, 9 max
<b>Escape hatch</b>			
<code>extra</code>	<code>()</code>	raw command strings appended verbatim	any flag from POV-Ray's <code>.ini</code> / CLI reference

Passing `none` suppresses the flag so POV-Ray's own default stays in force. The plugin always sets `+FN` (PNG), `-D` (no display), and `work_Threads=1`.

## 6. Resolution & quality

### 6.1. width / height

Control the output image size in pixels. Default is 800×600.

typst

```
#render("...", width: 1200, height: 900)
#render("...", width: 320, height: 240)
```

The scene's camera { right (image\_width/image\_height)\*x } uses these automatically.

### 6.2. quality

Integer 0–11 controlling which rendering features are enabled:

Level	Features enabled
0	Ambient light only
1	+ diffuse, pigment patterns
2	+ point lights, shadows
5	+ refractions (no refraction)
8	+ refractions, shadow transparency
9	+ full radiosity / media sampling ( <b>default</b> )

Lower quality renders faster — useful for draft iterations. Side-by-side on the same scene at 480×300 (no AA), median of 3 samples on the hardware listed in Section 16:

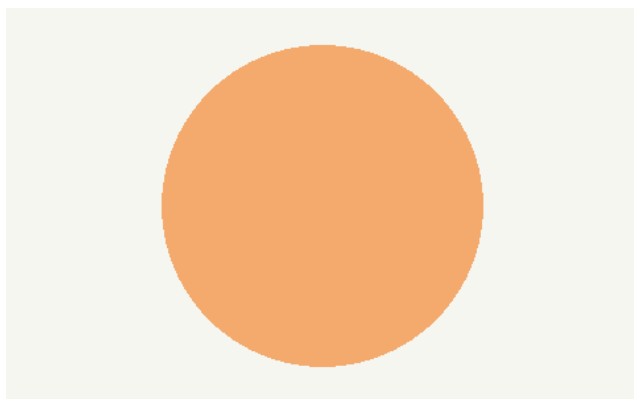


Figure 2: quality: 0 — ambient only, **0.81 s**.  
Useful while iterating on layout / framing.

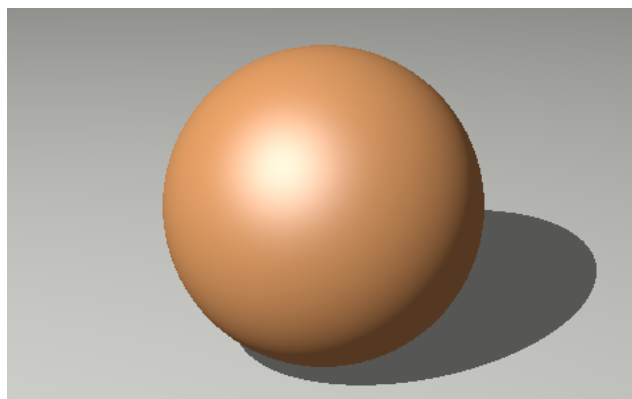


Figure 3: quality: 9, full lighting + shadows: **1.60 s**

Lighting-bound scenes benefit most; geometry-bound ones barely move.

## 7. Antialiasing

Enabled by default. Five kwargs control the behaviour:

kwarg	default	effect
antialias	true	master switch; false skips AA entirely
aa-threshold	0.3	colour delta that triggers extra samples; lower = smoother/slower
aa-method	2	1 fixed grid, 2 adaptive recursive, 3 generic oversampling (POV-Ray 3.8+) — not just edge AA, also suppresses moiré, image noise from jittered area lights, subsurface light transport, and micronormals. Parameterisation mirrors adaptive focal blur.
aa-depth	3	recursion depth; up to depth <sup>2</sup> samples per pixel
aa-jitter	true	randomise sub-pixel positions

## 8. Recipes

**Layout scaffolding.** When iterating on document layout you don't need photorealism. Drop quality, skip AA, and shrink resolution so each typst watch save lands in under a second — restore defaults once layout settles:

typst

```
#render("...", quality: 2, antialias: false, width: 320, height: 240)
```

**Publication quality.** For the camera-ready render, tighten the AA threshold and deepen recursion:

typst

```
#render("...", antialias: true, aa-threshold: 0.1, aa-depth: 4,
width: 1200, height: 900)
```

## 9. Transparent background

Set `output-alpha: true` and use `background { color rgbt <0,0,0,1> }` in the scene (`t = 1` means fully transparent). The result is an RGBA PNG that composites over the Typst page background.

typst

```
#pov("background { color rgbt <0, 0, 0, 1> }
camera { location <0, 1, -4> look_at 0 }
light_source { <4, 6, -5> rgb 1.3 }
sphere { 0, 1 pigment { rgb <0.2, 0.55, 0.9> } }",
output-alpha: true)
```

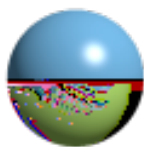
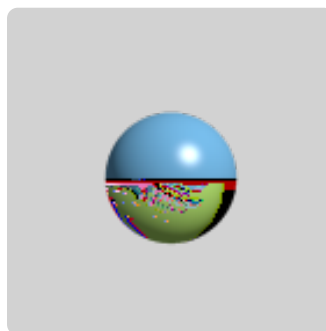


Figure 4: On white page



## 11. Typst-driven scene parameterisation

The first argument to `render()` is just a string — build it from any Typst values (loop index, palette entry, computed angle) to drive geometry, colour, or camera. The grid below is produced by a single map over three angles:

typst

```
#let palette = (rgb("#f26924"), rgb("#3ab573"), rgb("#4c6ef5"))
#let pov-rgb(c) = {
 let (r, g, b, ..) = c.components()
 "<" + str(r/100%) + ", " + str(g/100%) + ", " + str(b/100%) + ">"
}

#let orbit(theta, col) = (
 "global_settings { assumed_gamma 1.0 max_trace_level 2 }\n"
 + "background { color rgbt <0, 0, 0, 1> }\n"
 + "camera { location <" + str(5 * calc.cos(theta))
 + ", 2, " + str(5 * calc.sin(theta)) + "> look_at 0 angle 35 }\n"
 + "light_source { <4, 6, -4> rgb 1.3 }\n"
 + "torus { 1, 0.32 pigment { rgb " + pov-rgb(col) + " } "
 + "finish { specular 0.8 roughness 0.02 } rotate x*25 no_shadow }"
)

#grid(
 columns: (1fr, 1fr, 1fr), column-gutter: 12pt,
 ..((-90deg, 0), (-45deg, 1), (0deg, 2)).map(((a, i)) => render(
 orbit(a, palette.at(i)),
 width: 180, height: 180, quality: 2,
 antialias: true, aa-threshold: 0.7, aa-depth: 2, output-alpha: true,
)),
)
```



## 12. Tracing depth & advanced options

### 12.1. max-trace-level

Caps refraction / refraction ray bounces (POV-Ray default 5, overrides any in-scene `global_settings`). Drop to 2 for diffuse scenes; raise for glass-on-glass or hall-of-mirrors.

typst

```
#render("...", max-trace-level: 10)
```

## 12.2. extra (escape hatch)

Array of raw POV-Ray command strings appended verbatim — any flag from POV-Ray's .ini / CLI reference without a typed kwarg.

typst

```
#render("...", extra: ("wV3.8", "+HI"))
```

## 13. Examples

### 13.1. Constructive solid geometry

POV-Ray's signature feature: combine primitives via the boolean operators union, intersection, difference, and merge. Each takes any number of objects (including other CSG objects) and produces a new object that participates in further CSG. This is what lets a few dozen lines of SDL describe shapes that would take a mesh modeller hours.

The example below is built in three nested operations: intersection { box  $\cap$  sphere } rounds the cube's corners, union { 3 cylinders } collects three orthogonal rods, and difference { rounded-cube - rods } carves the rods out, exposing the rounded interior surfaces.

csg.pov

povray

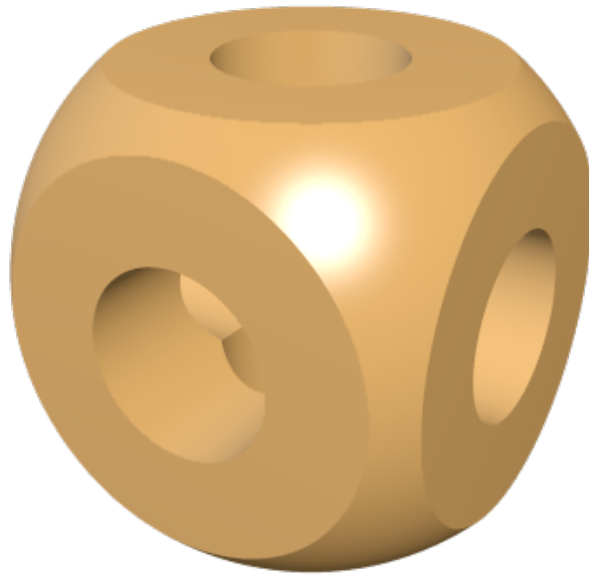
```
1 global_settings { assumed_gamma 1.0
 max_trace_level 2 }
2 background { color rgbt <0, 0, 0, 1> }
3 camera { location <2.6, 2.4, -4.2> look_at
 <0, 0, 0> angle 38 }
4 light_source { < 6, 9, -8> color rgb 1.4 }
5 light_source { <-4, 2, -2> color rgb <0.4,
 0.5, 0.95> shadowless }
6
7 #declare RoundedCube = intersection {
8 box { -<1,1,1>, <1,1,1> }
9 sphere { 0, 1.32 }
10 }
11
```

povray

```
12 #declare Bores = union {
13 cylinder { -1.2*x, 1.2*x, 0.42 }
14 cylinder { -1.2*y, 1.2*y, 0.42 }
15 cylinder { -1.2*z, 1.2*z, 0.42 }
16 }
17
18 difference {
19 object { RoundedCube }
20 object { Bores }
21 pigment { rgb <0.96, 0.55, 0.18> }
22 finish { ambient 0.12 diffuse 0.55
 specular 1.0 roughness 0.01 }
23 no_shadow
24 }
```

typst

```
#render(read("csg.pov"), output-alpha: true)
```



*Rounded cube with three orthogonal cylindrical bores — intersection, union, and difference in one object.*

## 13.2. Quaternion Julia fractal

Iterate the map

$$z_{n+1} = z_n^2 + c, \quad z \in \mathbb{H}, \quad c = -0.083 - 0.83j - 0.025k$$

over the quaternions  $\mathbb{H}$ . The rendered set is the boundary between starting points  $z_0$  whose orbit stays bounded and those that escape to infinity (bailout at  $n = 8$ ). A 3D slice of the 4-dimensional Julia set. See [Wikipedia on quaternion Julia sets](#).

julia.pov

povray

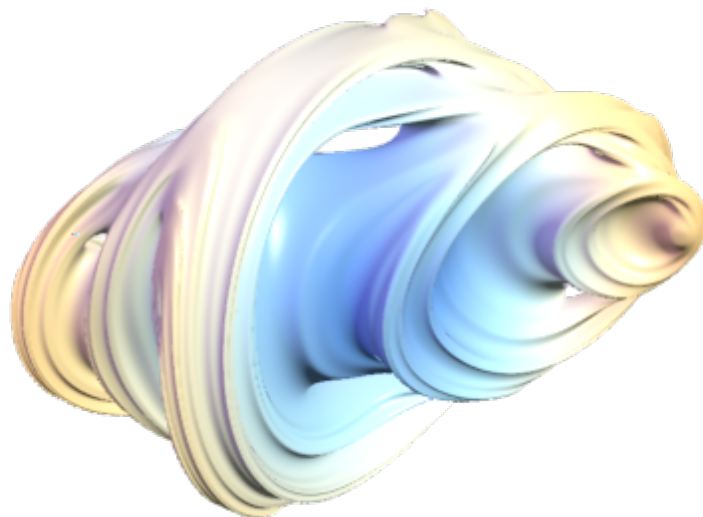
```
1 global_settings { assumed_gamma 1.0
 max_trace_level 2 }
2 background { color rgbt <0, 0, 0, 1> }
3
4 camera { location <0, 2, -4.5> look_at <0,
 0, 0> angle 38 }
5 light_source { <4, 6, -6> color rgb 1.6 }
6 light_source { <-5, 3, -2> color rgb <0.5,
 0.3, 0.8> shadowless }
7
8 julia_fractal {
9 <-0.083, 0.0, -0.83, -0.025>
10 quaternion
11 sqr
12 max_iteration 8
13 precision 50
14 slice <0, 0, 0, 1>, 0.15
15
16 pigment {
```

povray

```
17 spherical
18 color_map {
19 [0.00 rgb <0.95, 0.55, 0.18>]
20 [0.30 rgb <0.95, 0.78, 0.40>]
21 [0.55 rgb <0.45, 0.80, 0.95>]
22 [0.80 rgb <0.10, 0.30, 0.80>]
23 [1.00 rgb <0.05, 0.06, 0.25>]
24 }
25 scale 1.6
26 }
27 finish {
28 ambient 0.08
29 diffuse 0.85
30 specular 0.4
31 roughness 0.020
32 }
33 rotate <-20, 30, 0>
34 no_shadow
35 }
```

typst

```
#render(read("julia.pov"), antialias: true, aa-threshold: 0.4, aa-depth: 2, output-alpha: true)
```



Quaternion Julia set for  $c = -0.083 - 0.83j - 0.025k$ , 3D slice at  $q_w = 0.15$ , coloured radially from origin.

### 13.3. Clebsch diagonal cubic

In Cartesian coordinates  $(x, y, z) \in \mathbb{R}^3$ , the Clebsch surface is the zero set of the symmetric cubic polynomial

$$81(x^3 + y^3 + z^3) - 189(x^2y + x^2z + xy^2 + y^2z + xz^2 + yz^2) + 54xyz + 126(xy + yz + zx) - 9(x^2 + y^2 + z^2) - 9(x + y + z) + 1 = 0$$

A smooth cubic surface with full tetrahedral symmetry containing exactly 27 real straight lines (Cayley–Salmon, 1849; Clebsch’s explicit coordinates, 1869). POV-Ray’s cubic { ... } takes the 20 polynomial coefficients directly and ray-marches the zero set via Sturm sequences — no mesh, no approximation. See [Wikipedia on the Clebsch surface](#).

clebsch.pov

povray

```
1 global_settings { assumed_gamma 1.0
 max_trace_level 2 }
2 background { color rgbt <0, 0, 0, 1> }
3
4 camera { location <1.7, 1.5, -5.4> look_at
 <0, 0, 0> angle 42 }
5
6 light_source { <-6, 10, -10> color rgb 1.6 }
7 light_source { < 8, 2, -2> color rgb
 <0.20, 0.30, 0.60> shadowless }
8
9 cubic {
10 < 81, -189, -189, -9, -189, 54, 126,
 -189, 126, -9,
11 81, -189, -9, -189, 126, -9, 81, -9,
 -9, 1 >
```

povray

```
12 bounded_by { sphere { 0, 2 } }
13 clipped_by { sphere { 0, 1.6 } }
14 pigment {
15 gradient y
16 color_map {
17 [0.00 rgb <0.05, 0.10, 0.45>]
18 [0.40 rgb <0.10, 0.40, 0.92>]
19 [0.75 rgb <0.45, 0.80, 0.98>]
20 [1.00 rgb <0.85, 0.95, 1.00>]
21 }
22 scale 3 translate <0, -1.5, 0>
23 }
24 finish { ambient 0.07 diffuse 0.85
 specular 0.5 roughness 0.025 }
25 no_shadow
26 }
```

typst

```
#render(read("clebsch.pov"), output-alpha: true)
```



*Clebsch diagonal cubic, clipped to a sphere; stepped vertical gradient pigment, the contour bands tracing the surface’s height curvature.*

### 13.4. Schoen gyroid

In Cartesian coordinates  $(x, y, z) \in \mathbb{R}^3$ , Schoen's gyroid is a **triply periodic minimal surface** — the zero set of the transcendental function

$$f(x, y, z) = \sin x \cos y + \sin y \cos z + \sin z \cos x = 0$$

It locally minimises area (mean curvature zero everywhere) and tiles  $\mathbb{R}^3$  with cubic periodicity. Discovered by Alan Schoen at NASA in 1970. POV-Ray renders it via an isosurface.

*gyroid.pov*

povray

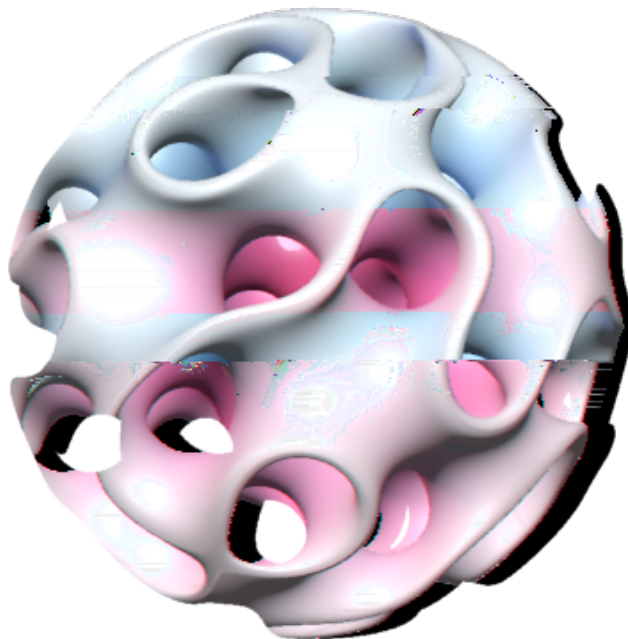
```
1 global_settings { assumed_gamma 1.0 max_trace_level
2 }
3 background { color rgbt <0, 0, 0, 1> }
4 camera { location <8, 6, -11> look_at <0, -0.3, 0>
5 angle 44 }
6 light_source { <6, 10, -8> color rgb 1.2 }
7 light_source { <-5, 1, -2> color rgb <0.22, 0.28,
8 0.50> shadowless }
9 #declare K = 2.0;
10 #declare R = 4;
11 #declare Slab = function(x,y,z) {
12 abs(sin(K*x)*cos(K*y) + sin(K*y)*cos(K*z) +
13 sin(K*z)*cos(K*x))
14 - (0.15 + 0.32 * (x*x + y*y + z*z) / (R*R))
15 }
16 #declare SphereSD = function(x,y,z) { sqrt(x*x +
17 y*y + z*z) - R }
18 #declare SmoothMax = function(a, b, k) {
19 (a + b + sqrt((a - b)*(a - b) + k*k)) / 2
20 }
```

povray

```
20 isosurface {
21 function { SmoothMax(Slab(x, y, z), SphereSD(x,
22 y, z), 0.25) }
23 contained_by { sphere { 0, R + 0.15 } }
24 threshold 0
25 accuracy 0.018
26 max_gradient 2
27 open
28 pigment {
29 spherical
30 color_map {
31 [0.00 rgb <0.85, 0.90, 0.95>]
32 [0.20 rgb <0.55, 0.80, 0.95>]
33 [0.50 rgb <0.20, 0.50, 0.92>]
34 [0.80 rgb <0.10, 0.25, 0.72>]
35 [1.00 rgb <0.15, 0.08, 0.40>]
36 }
37 scale R
38 finish { ambient 0.06 diffuse 0.88 specular 0.3
39 roughness 0.04 }
40 no_shadow
41 }
```

typst

```
#render(read("gyroid.pov"), antialias: true, aa-threshold: 0.7, aa-depth: 2, output-alpha: true)
```



*Schoen gyroid as a thin shell around the zero set; clipped to a sphere.*

### 13.5. Lissajous knot $8_{21}$

Parametric space curve

$$\mathbf{r}(t) = 0.75 \begin{pmatrix} \cos(3t + 0.1) \\ \cos(4t + 0.7) \\ \cos(7t) \end{pmatrix}, \quad t \in [0, 2\pi]$$

with frequencies (3, 4, 7) (pairwise coprime, phases (0.1, 0.7, 0)) — a prime knot with 8 crossings, classified as  $8_{21}$  in the [Rolfsen table](#). See [Wikipedia on Lissajous knots](#).

*knot.pov*

```
1 background { color rgbt <0, 0, 0, 1> }
2 global_settings { assumed_gamma 1.0
 max_trace_level 2 }
3
4 camera
5 {
6 location <0, 0, -25>
7 right <1,0,0> up <0,1,0>
8 look_at <0, 0, 0>
9 angle 3.5
10 }
11
12 light_source { <0, 20, -50> color rgb 1.6 }
13
14 #declare r_tube = 0.04;
15 #declare num_steps = 17;
16 #declare step_size = 1/num_steps;
17
18 sphere_sweep
19 {
```

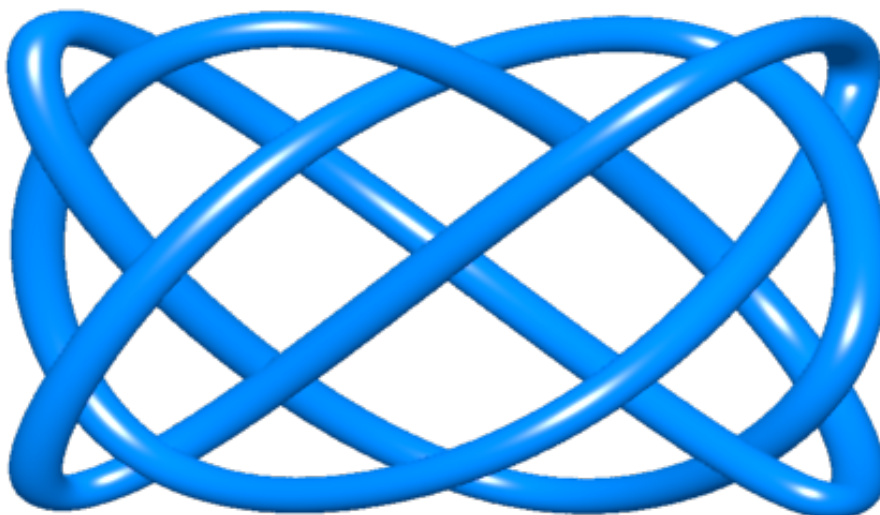
povray

```
20 b_spline num_steps+3,
21 #declare N = -1;
22 #while(N <= num_steps + 1)
23 #declare theta = 2 * pi * N * step_size;
24 < 0.75*cos(3*theta + 0.1),
25 0.75*cos(4*theta + 0.7),
26 0.75*cos(7*theta) >, r_tube
27 #declare N = N + 1;
28 #end
29 pigment { color rgb <0, 0.25, 1> }
30 finish
31 {
32 ambient 0.18
33 diffuse 0.75
34 specular 0.95
35 roughness 0.012
36 }
37 no_shadow
38 }
```

povray

typst

```
#render(read("knot.pov"), antialias: true, aa-threshold: 0.7, aa-depth: 2, output-alpha: true)
```



*Lissajous knot  $8_{21}$*

### 13.6. Hopf fibration

In geometry, the Hopf fibration gives a partition of the 3-dimensional sphere  $S^3$  by great circles. More precisely, it defines a fibred structure on  $S^3$ : the base space is the 2-dimensional sphere  $S^2$ , and the model fibre is a circle  $S^1$ . In particular, this means there exists a projection map  $p : S^3 \rightarrow S^2$  whose preimages of each point of  $S^2$  are circles. This structure was discovered by Heinz Hopf in 1931. See [Wikipedia on the Hopf fibration](#).

hopf.pov

povray

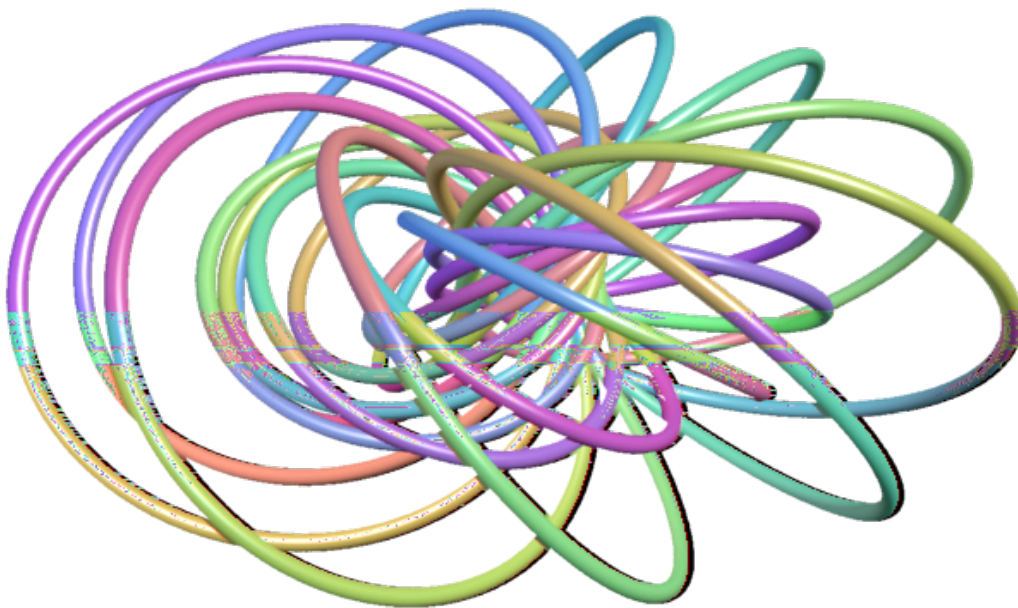
```
1 global_settings { assumed_gamma 1.0 max_trace_level
2 }
3 background { color rgbt <0, 0, 0, 1> }
4 camera { location <0, 4.6, -10.5> look_at <0,
5 -0.35, 0> angle 32 }
6 light_source { < 8, 12, -10> color rgb 1.4 }
7 light_source { <-6, 2, -3> color rgb <0.35, 0.45,
8 0.95> shadowless }
9 #declare r_tube = 0.06;
10 #macro HopfFiber(theta, phi, hue)
11 #local st = sin(theta/2);
12 #local ct = cos(theta/2);
13 #macro F(psi)
14 <ct*cos(psi), st*cos(psi+phi), ct*sin(psi)> /
15 (1 - st*sin(psi+phi))
16 #end
17 #local P0 = F(0); #local P1 = F(2*pi/3); #local
18 P2 = F(4*pi/3);
19 #local n = vcross(P1-P0, P2-P0);
20 #local a2 = vdot(P2-P1, P2-P1);
21 #local b2 = vdot(P0-P2, P0-P2);
22 #local c2 = vdot(P1-P0, P1-P0);
23 #local Cn = (a2*(b2+c2-a2)*P0 + b2*(c2+a2-b2)*P1
24 + c2*(a2+b2-c2)*P2) / (4*vdot(n,n));
25 #local N = vnormalize(n);
```

povray

```
22 #local T = (abs(N.y) < 0.9 ? <0,1,0> : <1,0,0>);
23 #local E1 = vnormalize(vcross(T, N));
24 #local E3 = vcross(N, E1);
25 torus {
26 vlength(P0-Cn), r_tube
27 pigment { rgb <0.55+0.4*cos(2*pi*hue),
28 0.55+0.4*cos(2*pi*hue+2*pi/3),
29 0.55+0.4*cos(2*pi*hue+4*pi/3)> }
30 finish { ambient 0.1 diffuse 0.55 specular 0.8
31 roughness 0.02 }
32 matrix <E1.x,E1.y,E1.z, N.x,N.y,N.z,
33 E3.x,E3.y,E3.z, Cn.x,Cn.y,Cn.z>
34 no_shadow
35 }
36 #end
37 #declare T = array[2] { 2*pi/5, 3*pi/5 };
38 #local i = 0;
39 #while (i < 2)
40 #local j = 0;
41 #while (j < 10)
42 HopfFiber(T[i], 2*pi*j/10, j/10 + i/2)
43 #local j = j + 1;
44 #end
45 #local i = i + 1;
46 #end
```

typst

```
#render(read("hopf.pov"), antialias: true, aa-threshold: 0.7, aa-depth: 2, output-alpha: true)
```



20 Hopf fibres over two parallels of  $S^2$ , coloured by hue of base point.

### 13.7. Advanced lighting

POV-Ray ships advanced light-transport features — refraction with chromatic dispersion, photon-mapped caustics, radiosity for indirect illumination, area lights with soft shadows, and volumetric media. This example exercises one of them: a wavy water surface (index of refraction  $n = 1.33$ ) refracts incident sunlight by Snell's law, and the bent rays concentrate onto the pool floor as the iconic shimmering caustic mesh. POV-Ray computes this by Jensen's photon mapping (1996): a pre-pass shoots photons from the light, tracks each through the wave-perturbed water normals, and stores landing positions in a kd-tree; the render then gathers nearby photons at each surface point. See [Wikipedia on caustics](#) and [photon mapping](#).

caustic.pov

povray

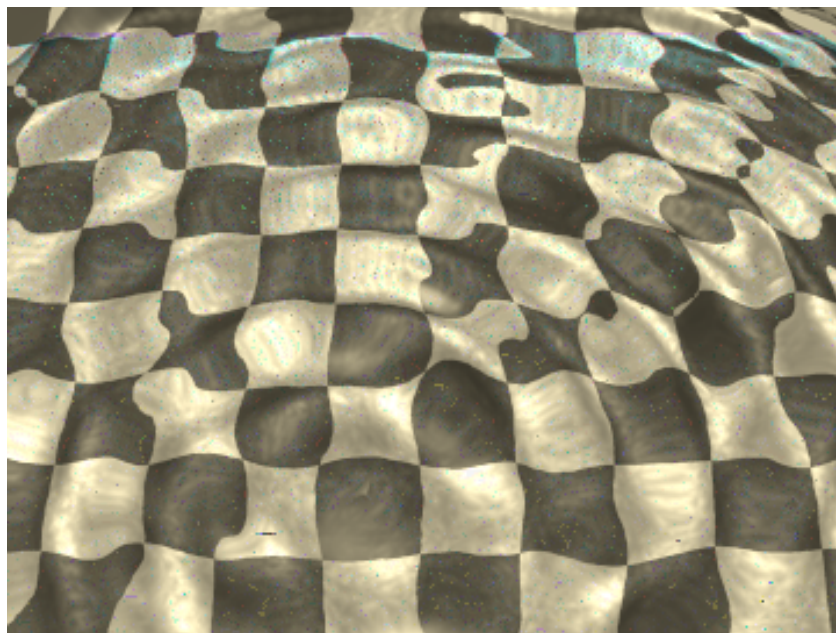
```
1 global_settings {
2 assumed_gamma 1.0
3 max_trace_level 3
4 photons { spacing 0.10 gather 3, 8 }
5 }
6 background { color rgbt <0, 0, 0, 1> }
7 camera { location <0, 5.5, -3.5> look_at <0, -1.6,
8 1.2> angle 38 }
9 light_source {
10 <-2, 15, -4> color rgb <1.0, 0.88, 0.55> * 2.4
11 photons { refraction on reflection off }
12 }
13
14 plane {
15 y, -2
16 pigment {
17 checker
18 color rgb <0.05, 0.05, 0.05>,
19 color rgb <0.29, 0.29, 0.29>
20 scale 0.6
21 }
```

povray

```
22 finish { ambient 0.04 diffuse 0.85 }
23 }
24
25 disc {
26 <0, 0, 0>, y, 4
27 pigment { rgbt <0.85, 0.92, 1.0, 0.94> }
28 finish {
29 ambient 0
30 diffuse 0.02
31 specular 0.3
32 roughness 0.012
33 reflection 0.04
34 }
35 normal {
36 ripples 0.6
37 frequency 2.5
38 turbulence 0.15
39 scale 1.2
40 }
41 interior { ior 1.33 }
42 photons { target refraction on collect off }
43 }
```

typst

```
#render(read("caustic.pov"))
```



Refractive caustics under a wavy water surface ( $n = 1.33$ ); photon mapping concentrates the sunlight onto the pool floor.

## 14. Debugging

POV-Ray parse-time errors surface as typst compile errors with a gcc-style location and source-line preview, courtesy of a parser hook in our build. `#error "msg"` is the supported authoring-time abort — test it from your scene to verify the diagnostic plumbing.

For visual orientation, draw a coordinate frame alongside the scene:

*debug-axes.pov*

```
1 global_settings { assumed_gamma 1.0 max_trace_level
2 }
2 background { color rgbt <0, 0, 0, 1> }
3
4 camera {
5 location <25, 25, 25>
6 look_at <0, 0.7, 0>
7 angle 8
8 }
9
10 light_source { <4, 6, -4> rgb 1.4 }
11
12 sphere {
13 <0, 1, 2>, 0.35
14 finish { specular 0.1 diffuse 0.1 }
15 }
16
17 #macro Axis(dir, col, len)
```

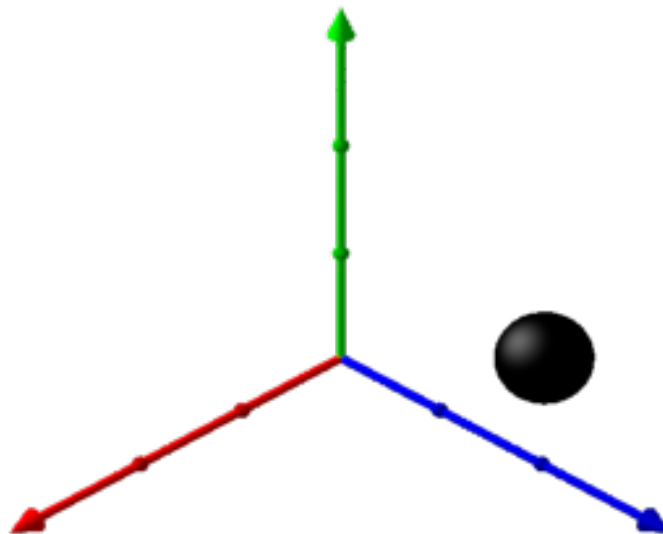
povray

```
18 cylinder { 0, len*dir, 0.035
19 pigment { rgb col } finish { ambient 0.2
20 diffuse 0.7 } no_shadow }
21 cone { len*dir, 0.10, (len+0.25)*dir, 0
22 pigment { rgb col } finish { ambient 0.2
23 diffuse 0.7 } no_shadow }
24 #local i = 1;
25 #while (i <= len)
26 sphere { i*dir, 0.06
27 pigment { rgb col*0.6 } finish { ambient 0.2
28 diffuse 0.7 } no_shadow }
29 #local i = i + 1;
30 #end
31 Axis(x, <1.0, 0.0, 0.0>, 3)
32 Axis(y, <0.0, 1.0, 0.0>, 3)
33 Axis(z, <0.0, 0.0, 1.0>, 3)
```

povray

typst

```
#render(read("debug-axes.pov"), antialias: true, aa-threshold: 0.7, aa-depth: 2, output-alpha: true)
```



*Origin-centred RGB axes (red = x, green = y, blue = z)*

## 15. Further reading and other examples

- **POV-Ray official documentation**  
<https://www.povray.org/documentation/>
- **Paul Bourke — Geometry**  
<https://paulbourke.net/geometry/>  
Marching cubes, minimal surfaces, Platonic solids, aperiodic tilings, most with POV-Ray source.
- **POV-Ray official — Object and Scene Files**  
[https://www.povray.org/resources/links/POV-Ray\\_Include\\_Macro\\_and\\_Object\\_Files/Object\\_and\\_Scene\\_Files/](https://www.povray.org/resources/links/POV-Ray_Include_Macro_and_Object_Files/Object_and_Scene_Files/)  
Community scene files and reusable macro libraries.
- **Friedrich Lohmüller — POV-Ray tutorial on shapes**  
[https://www.f-lohmueller.de/pov\\_tut/all\\_shapes/shapes000e.htm](https://www.f-lohmueller.de/pov_tut/all_shapes/shapes000e.htm)  
Visual walkthrough of every primitive and CSG idiom, from spheres and cylinders through text, isosurfaces, and height fields — each with its SDL snippet and rendered output.
- **Michael Scharrer — POV-Ray scenes**  
<https://mscharrer.net/povray/scenes/>  
22 algorithmic and fractal scenes, strong on CSG composition and mathematical visualisation; source available per scene.
- <http://dataduppedings.no/subcube/POV-Ray/index.html>, <http://bugman123.com/>, ...

## 16. Render times

Render times for each gallery scene, measured with an AMD Ryzen 9 9950X (Zen 5, 5.7 GHz).

Scene	Resolution	AA (threshold / depth)	Median time
basic.pov	480×300	—	2.3 s
csg.pov	480×360	—	3.4 s
julia.pov	420×315	0.4 / 2	4.3 s
clebsch.pov	480×360	—	4.1 s
gyroid.pov	520×390	0.7 / 2	19.3 s
knot.pov	480×320	0.7 / 2	15.1 s
hopf.pov	600×400	0.7 / 2	7.7 s
caustic.pov	360×270	—	13.5 s

The Gyroid takes longer because I tweaked it to look stylized/beautiful (clipped smoothly with a bounding sphere), and unfortunately raymarching isn't free. Refraction neither, same for caustic.pov.

## 17. Conclusion

POV-Ray has a lot of features, too many to show all of them here. Feel free to read its documentation to learn about its possibilities and look for examples online. Many wikipedia pages (in algebraic geometry, topology, and so on) use POV-Ray to visualize all sorts of things: <https://commons.wikimedia.org/wiki/Category:POV-Ray>